arXiv:2412.13786v2 [eess.AS] 28 Jan 2025

SongEditor: Adapting Zero-Shot Song Generation Language Model as a Multi-Task Editor

Chenyu Yang[1], Shuai Wang[1,3], Hangting Chen[*2], Jianwei Yu[*2], Wei Tan[2], Rongzhi Gu[2], Yaoxun Xu[4], Yizhi Zhou[6], Haina Zhu[5], Haizhou Li[1, 3]

[1]The Chinese University of Hong Kong, Shenzhen (CUHK-Shenzhen), Shenzhen, China
[2]Tencent AI Lab
[3]Shenzhen Research Institute of Big Data
[4]Tsinghua University
[5]X-LANCE Lab, Shanghai Jiao Tong University
[6]National Key Laboratory of Novel Software Technology, Nanjing University
chenyuyang2@link.cuhk.edu.cn, erichtchen@tencent.com, tomasyu@foxmail.com

Abstract

The emergence of novel generative modeling paradigms, particularly audio language models, has significantly advanced the field of song generation. Although state-of-the-art models are capable of synthesizing both vocals and accompaniment tracks up to several minutes long concurrently, research about partial adjustments or editing of existing songs is still under-explored, which allows for more flexible and effective production. In this paper, we present SongEditor, the first song editing paradigm that introduces the editing capabilities into language-modeling song generation approaches, facilitating both segment-wise and track-wise modifications. SongEditor offers the flexibility to adjust lyrics, vocals, and accompaniments, as well as synthesizing songs from scratch. The core components of SongEditor include a music tokenizer, an autoregressive language model, and a diffusion generator, enabling generating an entire section, masked lyrics, or even separated vocals and background music. Extensive experiments demonstrate that the proposed SongEditor achieves exceptional performance in end-to-end song editing, as evidenced by both objective and subjective metrics.

Demo — https://cypress-yang.github.io/SongEditor_demo/

Introduction

In recent years, applications for generating vocal music have achieved remarkable results. Compared with speech or accompanied singing voice, a song typically contains multiple sections interspersed with music-only segments like prelude, interlude, and postlude, which requires strict consistency and contextual coherence. Therefore, an ideal song generation system must be both structure-guided and capable of handling long content.

However, part of the generated song might be unsatisfactory in practice, but regenerating the entire sequence takes much time and computing resources, and potentially leads to further problems. Therefore, song editing is crucial as it allows for the refinement and enhancement of musical compositions. Moreover, there are instances where only a specific track, such as vocal or accompaniment, needs to be changed. How to rewrite the designated part of a song without destroying its contextual consistency and overall musicality, is a significant challenge for AI song composition.

Figure 1: SongEditor supports various song generation and editing tasks, involving creating complete songs from scratch and Infilling Editing. Accomp-to-song and vocal-to-song mean generating full songs based on partial input conditions such as accompaniments or vocals, respectively.

Currently, there are relatively few studies on end-to-end song generation in the academic field. State-of-the-art song generation platforms like Suno[1] are generally for commercial use and not open-source. Jukebox (Dhariwal et al. 2020) can synthesize songs in specific styles and genres, but it lacks more refined controls such as structural adjustments or specific instructions. To date, these approaches have primarily focused on pure generation and cannot be directly applied to editing scenarios.

Ding et al. (2024) proposed SongComposer, which supports various tasks like lyric-to-melody and melody-to-lyric, potentially allowing for some degree of song editing. However, SongComposer decomposes the song into lyrics and musical notes (MIDI), retaining no information about vocals such as timbre. Additionally, there is an unavoidable distor-

[*]Corresponding authors.

Copyright © 2025, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

[1]https://suno.com

tion between the original song and the extracted MIDI files during data preprocessing.

In this paper, we propose SongEditor, a general paradigm that enables partial editing in existing language-model-based song generation approaches. Since few techniques of song generation model are currently available, we first implement SongLM, a fundamental zero-shot song generation framework that can generate song sections up to two minutes long based on structure-guided lyrics and a 10-second excerpt from a reference song as an acoustic prompt. Subsequently, we make modifications in the architecture design and model training process, enabling the system to perform editing based on different types of contexts.

As shown in figure 1, in addition to generating songs from scratch, SongEditor can also regenerate specific periods and separate vocals or accompaniments. Worth noting is that the above two capabilities can be performed at the same time, allowing for the editing of even a short segment of vocals or accompaniment. The main contributions of this paper are summarized as follows:

- We propose SongEditor, a language model-based generation approach that handles long-content generation guided by user-defined lyrics and structures.
- SongEditor is among the first works designed not only to generate songs from scratch, but also to conduct sentence-level editing at the specific segment (*segment-wise*) and complete vocal or accompaniment tracks when given the rest (*track-wise*).
- A context-free editing method is employed that can effectively reduce the reliance on contextual lyrics and proves to be more suitable for long-content editing. Additionally, considering the complex composition of accompaniment, a series of measures are taken to ensure the consistency of editing segments and the naturalness of transitions.

We evaluated our model using both objective metrics and human evaluations. Results show that our model can generate songs with superior musicality and excellent consistency.

Related Work

Zero-Shot Speech Synthesis & Editing

Zero-shot Text-to-Speech (TTS) enables generating speech of unseen speakers based on their enrolled recordings. The current mainstream generation methods can be divided into language-modeling approaches and diffusion approaches. The former (Wang et al. 2023a; Du et al. 2024b; Borsos et al. 2023) proposes to generate discrete codec tokens (Défossez et al. 2022) autoregressive, while the latter (Shen et al. 2023; Chen et al. 2024b; Yang et al. 2024; Vyas et al. 2023; Wang et al. 2023b) leverages diffusion models to generate internal continuous features.

Moreover, some approaches have extensively explored the task of speech editing. For instance, VoiceCraft (Peng et al. 2024) proposes a token rearrangement method for the autoregressive generation procedure in neural codec language models. Apart from this, diffusion models (Vyas et al. 2023; Wang et al. 2023b; Du et al. 2024a) naturally possess the infilling editing ability in a non-autoregressive manner. These approaches have achieved a high quality of speech resynthesis. However, since the speech utterances used in these studies are relatively short and simple, they cannot be directly employed for song editing.

Music Generation

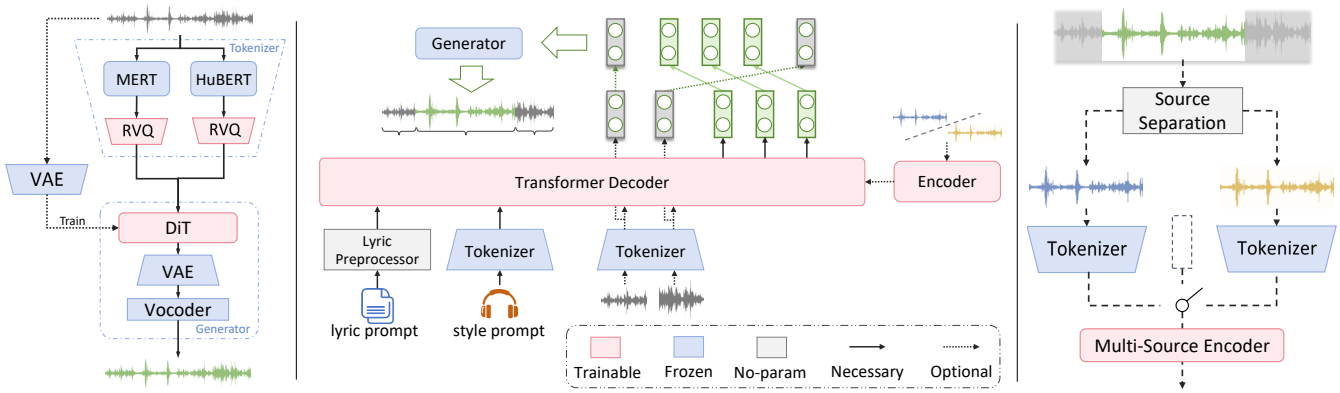
Recently, music generation has gained substantial attention from the research community. Compared to speech, music typically has a greater degree of variation in pitch, melody, tempo, and timbre. To facilitate user-friendly conditional generation without expert acknowledges, text-to-music has become exceptionally popular. For instance, MusicGen (Copet et al. 2023) and Mustango (Melechovsky et al. 2024) employ the T5 (Raffel et al. 2020) and Flan-T5 (Chung et al. 2024) text encoder to process the natural language prompts, and is capable of generating controllable music. Since the paired music and caption data is hard to collect, some approaches (Agostinelli et al. 2023; Lam et al. 2023; Chen et al. 2024a; Evans et al. 2024) leverage cross-modal alignment models (Huang et al. 2022; Elizalde et al. 2023; Wu et al. 2023) to project both music and text into the same embedding space instead. The condition vector derived from music can be used to train generative models in a self-supervised manner, while the vector from text is used for practical synthesis purposes. Some approaches incorporate musical signals, such as chromagrams (Copet et al. 2023) or beats (Lin et al. 2023) as auxiliary conditions. This additional information assists in making the generation process more controllable.

Several intriguing studies have also focused on music editing. For instance, Music ControlNet (Wu et al. 2024a) receives multiple types of musical signals to achieve aime-varying controls. MusicMagus (Zhang et al. 2024) aims to partially edit specific attributes of generated music by comparing the differences between old and new prompts. Meanwhile, VampNet (Garcia et al. 2023) is designed to complete the masked regions in acoustic token sequences with a non-autoregressive transformer.

Singing Voice Synthesis

Singing Voice Synthesis (SVS) aims to generate vocals based on both lyrics and musical scores. Benefiting from note pitches and durations, the SVS process becomes more controllable and easier to converge.

Conventional SVS systems primarily focus on incorporating phoneme-level melody controls into the generated acoustic representations. For instance, VISinger (Zhang et al. 2022a,b) adopts a methodology similar to VITS (Kim, Kong, and Son 2021), which learns the alignments between acoustic and textual units. TokSing (Wu et al. 2024b) discretizes raw audio using multiple self-supervised learning models and further enhances melody information during both the training and inference phases. Recent models (Liu et al. 2022; Hwang, Lee, and Lee 2025; He et al. 2023) have demonstrated promising performance for high-quality singing voice generation. HiddenSinger (Hwang, Lee, and Lee 2025), for instance, utilizes the latent diffusion



(a) Stage 1: Train tokenizer and generator.

(b) Stage 2: Train language model.

(c) Multi-source encoder for track-wise editing (optional).

Figure 2: The architecture of the proposed SongEditor framework. We train the DiT and RVQ jointly first and then the semantic language model. The multi-source encoder is exclusively used for track-wise editing.

model (Rombach et al. 2022) to synthesize regularized intermediate codec representations. Meanwhile, RMSSinger (He et al. 2023) proposes a DDPM-based method (Ho, Jain, and Abbeel 2020) that synthesizes voice using word-level coarse-grained musical scores, alleviating the heavy demand for manual annotations in real-world scenarios.

Base Model: SongLM

Before introducing the editing framework, we would first like to demonstrate the base system, SongLM, for our editing paradigm. Figure 2 presents the architectural framework of SongLM, which accepts multiple lyric sentences and a 10-second acoustic prompt as inputs. The primary components of this framework include a semantic tokenizer, a language model, and a diffusion-based generator. The semantic tokenizer compresses audio waveforms including the acoustic prompt, context, and separated tracks, into discrete semantic tokens. Subsequently, the language model generates semantic tokens in an autoregressive manner. The final waveform is reconstructed from the output sequence by the diffusion generator.

The training process is divided into two distinct phases. Initially, the tokenizer and diffusion generator are trained concurrently. Subsequently, the language model is trained using tokens generated by the tokenizer. During the second phase, both the tokenizer and the diffusion generator remain in a frozen state.

Lyric preprocessor The original data pairs only contain plain text and waveform $\{(X_i, W_i)\}_{i=1}^{L}$, where L is the number of lyrics, X_i is the i-th sentence and W_i is the corresponding waveform. It is worth noting that both waveforms and lyrics are consecutive, so they can be concatenated in sequence and W_i may be empty at non-vocal sections. However, relying solely on the text of lyrics is not enough for the complicated song generation task. In order to incorporate the structure information into the lyrics, a structure detector (Kim and Nam 2023) is applied. The category of structure indicators is shown in Appendix A.

Semantic Tokenizer Neural codec tokens have been widely used by previous music generation systems (Huang et al. 2022). However, we found that the quantized semantic tokens (Borsos et al. 2023; Agostinelli et al. 2023) derived from self-supervised training models exhibit a higher compression rate. In this paper, we propose a tokenizer consisting of two distinct branches with different pre-trained encoders: MERT (Li et al. 2024) and HuBERT (Hsu et al. 2021). The former focuses on accompaniment while the latter is for vocal. Two Residual Vector Quantizers (RVQ) (Zeghidour et al. 2021; Kumar et al. 2023) are appended to the end of each branch. And each quantizer has two layers, resulting in $K = 4$ tokens in each frame. The semantic token extraction can be formulated as:

$$Y = \mathrm{RVQ}\left(\mathrm{MERT}(W)\right) \oplus \mathrm{RVQ}\left(\mathrm{HuBERT}(W)\right), \quad (1)$$

where $Y \in \mathbb{Z}^{T \times K}$ is the discrete token sequence and T is the sequence length. W is the input waveform. $\oplus$ denotes the frame-by-frame concatenation. During the training phase, only the RVQ layers are updated using a commitment loss (van den Oord, Vinyals, and kavukcuoglu 2017):

$$\mathbb{L}_{rvq} = \sum^{K} \left(||sg(e_k) - z_k||_2^2 + ||e_k - sg(z_k)||_2^2 \right), \quad (2)$$

where $sg(\cdot)$ is the stop-gradient operation, z_k is the input latent vector and e_k is the nearest codebook entry.

Decoder-only Language Model The condition tensors and semantic token sequence are first concatenated together and then fed into a decoder-only transformer. Each transformer layer contains a causal self-attention mechanism with Rotary Position Embeddings (RoPE) (Su et al. 2024) and a feed-forward block.

Diffusion Generator We utilize the Latent Diffusion Model (LDM)(Rombach et al. 2022) as the generator, comprising a diffusion model, a variational autoencoder (VAE)(Kingma and Welling 2013), and a vocoder. We replace the conventional U-Net (Ronneberger, Fischer, and

Brox 2015) backbone with DiT (Peebles and Xie 2023), which conditions on semantic tokens and applies forward and reverse processes on the latent vectors produced by VAE (van den Oord, Vinyals, and kavukcuoglu 2017). The VAE then converts latent vectors into a Mel spectrogram, which is subsequently transformed into a waveform by the HifiGAN (Kong, Kim, and Bae 2020) vocoder.

SongEditor

In this section, we will present SongEditor, a novel framework that integrates editing capabilities into a base model. SongEditor enables two types of song editing: segment-wise, which modifies the vocals and accompaniment within a song segment simultaneously, and track-wise, which allows independent editing of the vocals or accompaniment. Importantly, these approaches can be combined, empowering the flexibility to edit song segments on a single track or holistically. In the following, we will detail how SongEditor implements these editing functionalities.

Long-Content Segment-Wise Editing

For segment-wise editing, we first discuss leveraging contextual information for the segment being edited. Then, we introduce a rearrangement operation similar to VoiceCraft (Peng et al. 2024), applied to the semantic token sequence. To achieve a more natural transition effect, we propose a force-smoothing strategy during training and score-based candidate selection during inference.

Context Selection The waveform segment to be edited is defined as $W_{[L_A:L_B]} = [W_{L_A}, W_{L_A+1}, \cdots, W_{L_B}]$, where $1 \leq L_A \leq L_B \leq L$. Here, L_A and L_B denote the start and end sentences. It is worth noting that L_A and L_B may be equal to 1 or L, allowing SongEditor to also perform the continuation or generation task. During the training phase, the preceding and following contexts $y_{[1:A)}$ and $y_{(B:T]}$ are directly cut from the token sequence Y, where A and B are the start and end indexes of frames. During inference, these contexts are extracted by the semantic tokenizer from the waveforms $W_{[1:L_A)}$ and $W_{(L_B:L]}$ respectively, with an overlap of 1 second to avoid edge effects. The overlapped area is subsequently removed after tokenization. For the lyrics input, only the edited sentences are retained, represented as $X = [X_{L_A}, X_{L_A+1}, \cdots, X_{L_B}]$, which we refer to as a lyric context-free strategy. The choice of not including the lyric context is based on two observations: 1) The contextual connection between lyric sentences is not particularly tight. 2) A context-free approach can achieve more precise control over resynthesized vocals, as contextual lyrics can sometimes lead to incorrect vocal generation.

Additionally, we have observed that the context-free strategy is more suitable for practical applications, such as consecutive generation for songs that exceed the maximum length of training samples. By eliminating the need for context lyrics, the language model can automatically generate a much longer song one step at a time (Agostinelli et al. 2023), without requiring human interception and annotation of the preceding context at each step. This approach enhances both efficiency and usability in practical scenarios.

Rearrangement Assuming the original sequence consists of three segments: $Y = [y_{[1:A)}, y_{[A:B]}, y_{(B:T]}]$, during the training phase, the editing segment is moved to the end of the sequence, and a ⟨SEP⟩ token is appended at the end of each segment. Thus, the rearranged sequence should be $Y^{re} = [y_{[1:A)}, y_{<s>}, y_{(B:T]}, y_{<s>}, y_{[A:B]}, y_{<s>}]$, where $y_i = (y_{i,k})_{k=1}^{K}$ and $y_{<s>}$ denotes the embedding of the ⟨SEP⟩ token.

A delayed pattern (Copet et al. 2023) is further applied to rearrange the K RVQ tokens. Tokens of the k-th quantizer are moved $(k-1)$ timesteps backward and extra $(K-1)$ empty frames are appended to each segment in order to prevent overlap of quantizers. Then tokens at the same timestep are stacked together. Consequently, the final length of the language model input should be $(T+3K)$. Because the lyrics of contexts have been discarded, only the training loss of the editing segment is calculated as $\mathbb{L}_{ce} = -\log\left(P_\theta^{[A,B]}\right)$, where

$$P_\theta^{[A,B]} = P_\theta\left(y_{[A:B]} | y_{[1:A)}, y_{(B:T]}, X, Y^{sty}\right) \tag{3}$$

$$= \prod_{i=A}^{B}\prod^{K} p_\theta\left(y_{i,k} | \forall y_{m,n}, X, Y^{sty}\right), \tag{4}$$

if $m \in (0, A) \cup (B, T)$ or $n + m < i + k$.

Force-Smoothing Training Due to the presence of melody and rhythm, listeners tend to be more sensitive and strict about interruptions in music compared to speech. However, ensuring smoothness is more challenging for music editing, especially at the endpoint of the editing segment. To tackle this issue, we propose a force-smoothing strategy. During the training phase, the model is enforced to predict additional λ frames with a probability of 0.1 for each step even after the editing segment has ended. These frames are directly copied from the beginning of the following context $y_{(B:T]}$, and the $P_\theta^{[A,B]}$ for loss calculation is replaced by $P_\theta^{[A,B+\lambda]}$. During inference, the model will first greedily determine whether to stop prediction at the current step. As long as the probability of ⟨eos⟩ is the largest, the model will immediately output ⟨eos⟩ token. Otherwise, a top-k sampling method will be employed to select the next token randomly from the remaining options. this sampling strategy can effectively alleviate the over-writing issue caused by force-smoothing during training.

Score-Based Candidate Selection To achieve better transitions, a score-based candidate selection (Jiang et al. 2023) is applied during inference. Specifically, we first generate an initial candidate $\hat{y}^1_{[A:B]}$ and resynthesize the last 3 seconds for $(N-1)$ times, resulting in a set of N candidates $\hat{Y}_{[A:B]} = \{\hat{y}^1_{[A:B]}, \hat{y}^2_{[A:B]}, \cdots, \hat{y}^N_{[A:B]}\}$. Each candidate is used as a new prefix to predict the subsequent λ frames. The score of each candidate can be represented as the log-likelihood of $y_{(B:B+\lambda]}$. The candidate with the highest score is ultimately selected, which can be formulated as:

$$y = \arg\max_{\hat{y} \in \hat{Y}_{[A:B]}} P_\theta\left(y_{(B:B+\lambda]} | y_{[1:A)}, \hat{y}, X, Y^{sty}\right). \tag{5}$$

Multi-Source Track-Wise Editing

Furthermore, we explore the integration of vocal and accompaniment completion into the model. As depicted in Figure 2, SongEditor utilizes either the separated vocal or accompaniment track as auxiliary context to complete the other. The provided track is processed through a gated multi-source encoder and incorporated into the language model. Details of this process are introduced below.

Source Separation The Band-Split RNN (BS-RNN) (Yu et al. 2023; Luo and Yu 2023) is adopted as source separation module for track-wise editing. BS-RNN splits the spectrogram into multiple subbands and performs bidirectional LSTM layers across both subbands and frames. Given that music typically has a higher sample rate and a broader frequency range, BS-RNN is exceptionally suitable for music source separation. We leverage BS-RNN to separate vocals from the mixture, leaving the remaining as accompaniments.

During the training phase, a white noise $\epsilon \sim \mathbb{N}(0, \sigma^2)$ is added to the separated vocals in order to prevent the potential leakage of remaining music components after separation (Donahue et al. 2023). In this paper, σ is set to 0.01, resulting in a signal-to-noise ratio (SNR) of 40dB.

Multi-Source Encoder As shown in Figure 2, to be compatible with various conditions, both vocal and accompaniment share the same tokenizer and embedding layers. The token embedding of each frame can be represented as:

$$c_i = \begin{cases} \sum^K c_{i,k} + t & t \in \{t^M, t^V\} \\ t & t = t^{\varnothing}. \end{cases} \tag{6}$$

where t^M, t^V, and $t^{\varnothing}$ are special embeddings indicating the type of source, respectively accompaniment, vocal, and none. $c_{i,k} \in \mathbb{R}^D$ is the quantizer embedding and D is the dimension of embedding vectors. If one source is provided, the sequence should be $c = (c_t)_{t=1}^T \in \mathbb{R}^{T \times D}$; otherwise, $c \in \mathbb{R}^{1 \times D}$. The sequence is passed through a multi-layer transformer encoder with RoPE and injected into each transformer decoder layer via cross-attention.

Experimental Settings

Datasets

For the training of SongEditor, a large-scale dataset with approximately 700K songs was used, which sums up to 50K hours. A data cleaning pipeline similar to (Yu et al. 2024) was adopted to filter out the low-quality samples and correct the timestamp of lyrics in the preprocessing process. BSRNN is initially employed to extract vocals. Pyannote (Bredin 2023) and WhisperX (Bain et al. 2023) toolkits are then leveraged for voice activity detection (VAD) and singing voice recognition (SVR) respectively. Lyrics with serious mismatches will be discarded. The time boundary will be corrected based on the VAD results and two adjacent lyric sentences with a short gap will be merged.

For objective evaluation, we assembled a test set of 200 randomly selected samples. Each sample ranges from 90 to 120 seconds in length and contains at least one complete verse and chorus. A 10-second prompt is extracted from other parts of the song. For the song editing task, we randomly mask a region, which may be located in the middle, beginning, or end of the song. For subjective evaluation, we randomly select 15 samples with manual correction of annotations, and two recordings are generated based on each sample, which are then assessed by expert musicians.

Method	PER(%)↓	FAD↓	Musicality↑	Quality↑
GT(restore)	5.51	0.86	3.63	3.59
SongLM	20.63	**1.99**	2.95	3.06
SongEditor	**18.33**	2.24	**3.02**	**3.19**

Table 1: Evaluation results of baseline and our proposed method for song generation.

Evaluation Metrics

To conduct a comprehensive comparison, we evaluated three different models: SongLM, SongEditor, and SongEditor+. SongLM serves as our baseline and can only generate songs from scratch. SongEditor supports segment-wise editing, while SongEditor+ is capable of both segment- and track-wise editing. Details of them are presented in Appendix B.

The proposed models are evaluated in terms of both objective and subjective metrics. For objective evaluation, both Phoneme Error Rate (PER) and Fréchet Audio Distance (FAD) (Kilgour et al. 2019) are employed. To calculate PER, the vocal is first extracted by BSRNN. Then the Whisper-large-v2 is utilized for speech recognition. All punctuation and structure tokens in the lyrics have been removed. The FAD score is computed based on the internal representation from the last layer of MERT-95M, and the fadtk[2] toolkit is used for calculating statistics and comparison. For editing tasks, only the editing segments are taken into consideration.

For human evaluation, we conduct a mean opinion score (MOS) listening test. Specifically, we employ 30 listeners to rate each audio sample on a scale from 1 to 5 across five aspects: musicality, quality, coherence, smoothness, and intelligibility. Coherence measures the degree of consistency between the editing part and context. Smoothness refers to the naturalness of transitions. These two scores are only considered for the segment-wise editing task.

In our experiments, we observe that tracks restored from the original audio sometimes significantly reduce the performance of the source separation module, resulting in an abnormally high PER. To adjust the quality of vocals more precisely, we also ask the listeners to assess whether the generated vocals sound clear and match the given lyrics (intelligibility) for a track-wise editing task.

Results and Analysis

Song Generation

Table 1 compares the performance of our proposed methods to the baseline for lyric-to-song generation. Additionally, we demonstrate the results of directly restoring audio from ground truth semantic tokens ("restore"), which represents the upper bound of this framework. Incorporating

[2]https://github.com/microsoft/fadtk

Method	Context-Free	Objective		Subjective (MOS)			
		PER(%)↓	FAD↓	Musicality↑	Quality↑	Coherence↑	Smoothness↑
VoiceCraft (Peng et al. 2024)	✗	35.23	1.34	3.03	3.17	3.48	**3.63**
SongEditor	✓	**20.82**	**1.30**	**3.25**	**3.39**	**3.82**	3.53
- CS	✓	-	-	-	-	-	3.50
- CS & FS	✓	25.68	1.37	3.13	3.23	3.54	3.43

Table 2: Evaluation results of SongEditor for segment-wise editing. "CS" represents the score-based candidate selection. Since it only affects the local smoothness, other metrics are dismissed. "FS" represents force-smoothing.

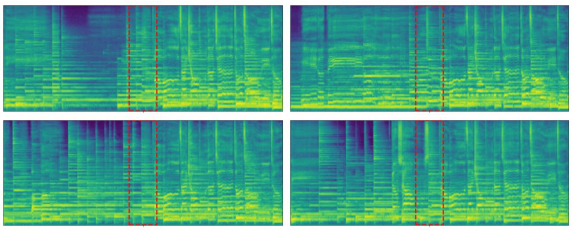
segment-wise editing capability does not degrade the performance of SongLM. Instead, a slight improvement can be observed in terms of subjective MOS scores and PER.

Segment-Wise Editing

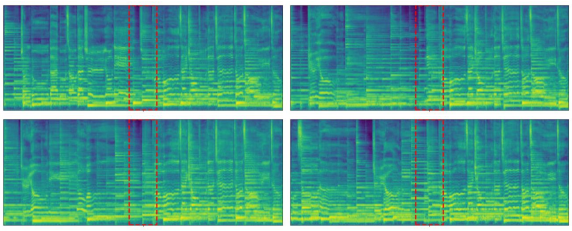
Table 2 reports the result of SongEditor for segment-wise editing. To demonstrate the advantage of the context-free strategy used in our model, we adopt VoiceCraft, a context-based approach for speech editing as our baseline. VoiceCraft takes the complete lyrics of both contexts and modified sentences as input. During training, the start and end points are randomly selected. The proposed SongEditor significantly outperforms the baseline in PER. We further find that sentence-level mistakes, such as repeating or missing, are more likely to happen around the editing segments. We believe these mistakes are caused by the failure to align the boundary of context audio and lyrics. In most of the subjective metrics, especially the coherence score, SongEditor also surpasses the baseline, which proves that the context-free strategy is more suitable for such a long-content editing task. One notable observation is that the smoothness score deteriorates for our proposed model. We attribute this decline to the potential inaccuracies in the annotation of temporal boundaries within the training data.

An ablation study was conducted to verify the effectiveness of force-smoothing training. We retrained SongEditor for the same number of steps without force-smoothing. As shown in Table 2, all metrics declined to varying degrees. This may suggest that force-smoothing training not only enables the model to produce smoother transitions but also helps it learn long-term dependencies in the following context. Additionally, with the score-based candidate selection during inference, SongEditor can achieve a higher smoothness score even without further training.

Figure 3 compares the Mel spectrograms of the transition areas. We randomly generated several samples with the same context. Since the autoregressive generation progresses from left to right, the transition from the preceding context to the edited segment is relatively straightforward, so we primarily focus on transitions from generated to subsequent audio. A 10-second audio clip is captured around the end point of each sample. As shown in Figure 3, after applying force-smoothing and candidate selection, the Mel spectrograms of the generated samples appear smoother, with high-energy bands becoming more continuous. In contrast, distinct discontinuities or mismatches can be observed at the transition points without force-smoothing.



(a) Smoothing.



(b) No smoothing.

Figure 3: Mel spectrograms of transitions. The center of the red box is the transition point. The left half is generated while the right half is restored from ground truth.

Track-Wise Editing

Moreover, we evaluate SongEditor with cross-attention layers on the track-wise editing task. In this section, we primarily use the Intelligibility MOS score to assess the correctness of the generated vocals, as combining restored and generated tracks can result in abnormal PER outcomes. Our experiments investigate the influence of different source tracks by testing various configurations: vocals only (vocal-to-song), accompaniment only (accomp-to-song), and neither of them. Track-wise information is integrated into both the generation and segment-wise editing tasks.

As shown in Table 3, both vocals and accompaniments boost performance in terms of musicality and FAD scores. The vocal track plays a dominant role in track-wise editing, which is expected since vocals often serve as the backbone of a song, directly conveying crucial elements such as rhythm and emotion. Additionally, listeners naturally focus more on vocals. Conversely, generating vocals from accompaniments presents greater challenges, as the model must maintain precise rhythm synchronization with accompaniments, resulting in a slight decrease in the intelligibility score as a trade-off. Notably, the FAD score for vocal-to-song generation from scratch is considerably high (2.71) but

input V	input A	w/ context	Objective PER(%)↓	Objective FAD↓	Subjective (MOS) Musicality↑	Quality↑	Coherence↑	Smoothness↑	Intelligibility↑
✓	✗	✗	-	2.71	**3.38**	**3.54**	-	-	**4.35**
✗	✓	✗	-	**1.29**	2.83	2.90	-	-	3.74
✗	✗	✗	19.30	2.61	2.68	2.92	-	-	3.98
✓	✗	✓	-	**1.13**	**3.56**	**3.66**	**4.19**	**3.99**	**4.33**
✗	✓	✓	-	1.17	3.21	3.06	3.48	3.72	3.53
✗	✗	✓	23.72	1.32	2.93	3.06	3.33	3.47	3.57

Table 3: Performance of SongEditor+ with different source tracks. "V" is the abbreviation of vocal and "A" is accompaniment. The upper part compares the performance for pure generation, while the lower part introduces additional contextual information.

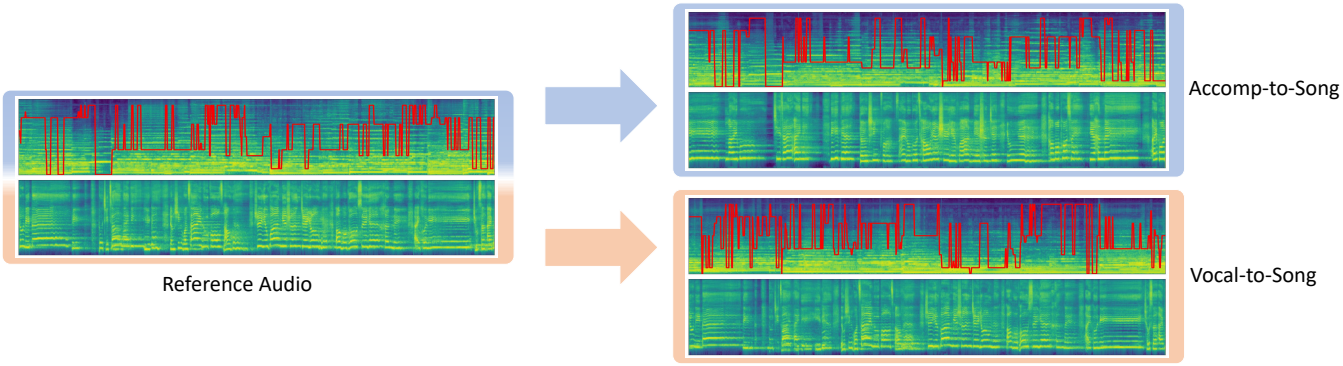


Figure 4: An example of track-wise editing. The Mel spectrogram above corresponds to the separated accompaniment, while the below corresponds to the vocal. Since the spectrogram of accompaniment is more complex and difficult to identify, its chroma change trend is plotted (red line).

drops significantly when contextual information is provided (1.13). This suggests that the model can learn more detailed accompaniment features when contexts are available.

Figure 4 illustrates the result of music separation for both reference and generated audio. Our goal is to study the consistency of the regenerated audio with the original source input. Specifically, we first separate the reference audio into vocals and accompaniment, then use SongEditor+ to independently complete each track. As shown in the figure, the generated vocal spectrogram closely resembles the input in the vocal-to-song task. Conversely, for accomp-to-song, while the restored chromagram generally remains consistent, some differences can be observed in both the chromagram and spectrogram. We believe that vocals are relatively independent and easier to compress, while accompaniments often rely on vocals and contain more detailed information.

Consecutive and Multi-Singer Story Mode

As highlighted in Section 4, the context-free strategy demonstrates significant advantages in practical applications. In this section, we explore generating full songs longer than two minutes through an iterative round-by-round process, referred to as Story Mode, following Agostinelli et al. (2023). The complete lyrics are pre-divided into several sections. During each round, only the lyrics of the current section are provided, while a fixed-length token sequence is automatically extracted from the previous output as the prefix. No manual intervention is required throughout the process.

Specifically, we adopt two task settings: full song generation and multi-singer story mode. The former aims to generate a full song from intro to outro, without changing the style prompt. This task focuses on the coherence across rounds, and we advance with a stride of 60 seconds. For the latter, we use two different style prompts, typically one for male and one for female. Verse and chorus sections are generated with alternate prompts. In order to prevent the emergence of vocal in the prefix, a short instrumental segment is generated at the end of each round and the stride length is set to 5 seconds. Experiments demonstrate that our model can generate full songs with smooth transitions and coherent content. By slightly modifying the task settings, it can also achieve multi-singer generation with variational style prompts. Audio samples are presented in the demo page.

Limitations and Ethic Discussion

Limitations: SongEditor currently regenerates specified sentences without providing additional control over the editing segment. Furthermore, there is no explicit decoupling of tracks at the semantic level, which could enhance interpretability and improve the distinction.

Ethics: SongEditor is an innovative tool that helps either a professional musician or a passionate enthusiast to create their own songs. However, we also fully acknowledge the potential ethical risks. We ensure that our training data does not infringe on any copyright, and only public-domain melodies are used for inference.

Acknowledgments

This work was supported by Shenzhen Science and Technology Program (Shenzhen Key Laboratory Grant No. ZDSYS20230626091302006) and Shenzhen Science and Technology Research Fund (Fundamental Research Key Project Grant No. JCYJ20220818103001002).

References

Agostinelli, A.; Denk, T. I.; Borsos, Z.; Engel, J.; Verzetti, M.; Caillon, A.; Huang, Q.; Jansen, A.; Roberts, A.; Tagliasacchi, M.; et al. 2023. Musiclm: Generating music from text. *arXiv preprint arXiv:2301.11325*.

Bain, M.; Huh, J.; Han, T.; and Zisserman, A. 2023. WhisperX: Time-Accurate Speech Transcription of Long-Form Audio. *INTERSPEECH*.

Borsos, Z.; Marinier, R.; Vincent, D.; Kharitonov, E.; Pietquin, O.; Sharifi, M.; Roblek, D.; Teboul, O.; Grangier, D.; Tagliasacchi, M.; et al. 2023. Audiolm: a language modeling approach to audio generation. *IEEE/ACM transactions on audio, speech, and language processing*, 31: 2523–2533.

Bredin, H. 2023. pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe. In *Interspeech*, 1983–1987.

Chen, K.; Wu, Y.; Liu, H.; Nezhurina, M.; Berg-Kirkpatrick, T.; and Dubnov, S. 2024a. Musicldm: Enhancing novelty in text-to-music generation using beat-synchronous mixup strategies. In *ICASSP*, 1206–1210.

Chen, Y.; Niu, Z.; Ma, Z.; Deng, K.; Wang, C.; Zhao, J.; Yu, K.; and Chen, X. 2024b. F5-tts: A fairytaler that fakes fluent and faithful speech with flow matching. *arXiv preprint arXiv:2410.06885*.

Chung, H. W.; Hou, L.; Longpre, S.; Zoph, B.; Tay, Y.; Fedus, W.; Li, Y.; Wang, X.; Dehghani, M.; Brahma, S.; et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70): 1–53.

Copet, J.; Kreuk, F.; Gat, I.; Remez, T.; Kant, D.; Synnaeve, G.; Adi, Y.; and Defossez, A. 2023. Simple and Controllable Music Generation. In *NeurIPS*, volume 36, 47704–47720.

Défossez, A.; Copet, J.; Synnaeve, G.; and Adi, Y. 2022. High fidelity neural audio compression. *arXiv preprint arXiv:2210.13438*.

Devlin, J.; Chang, M.-W.; Lee, K.; and Toutanova, K. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Dhariwal, P.; Jun, H.; Payne, C.; Kim, J. W.; Radford, A.; and Sutskever, I. 2020. Jukebox: A generative model for music. *arXiv preprint arXiv:2005.00341*.

Ding, S.; Liu, Z.; Dong, X.; Zhang, P.; Qian, R.; He, C.; Lin, D.; and Wang, J. 2024. Songcomposer: A large language model for lyric and melody composition in song generation. *arXiv preprint arXiv:2402.17645*.

Donahue, C.; Caillon, A.; Roberts, A.; Manilow, E.; Esling, P.; Agostinelli, A.; Verzetti, M.; Simon, I.; Pietquin, O.; Zeghidour, N.; et al. 2023. Singsong: Generating musical accompaniments from singing. *arXiv preprint arXiv:2301.12662*.

Du, C.; Guo, Y.; Shen, F.; Liu, Z.; Liang, Z.; Chen, X.; Wang, S.; Zhang, H.; and Yu, K. 2024a. UniCATS: A unified context-aware text-to-speech framework with contextual vq-diffusion and vocoding. In *AAAI*, volume 38, 17924–17932.

Du, C.; Guo, Y.; Wang, H.; Yang, Y.; Niu, Z.; Wang, S.; Zhang, H.; Chen, X.; and Yu, K. 2024b. VALL-T: Decoder-Only Generative Transducer for Robust and Decoding-Controllable Text-to-Speech. *arXiv preprint arXiv:2401.14321*.

Elizalde, B.; Deshmukh, S.; Al Ismail, M.; and Wang, H. 2023. Clap learning audio concepts from natural language supervision. In *ICASSP*, 1–5.

Evans, Z.; Parker, J. D.; Carr, C.; Zukowski, Z.; Taylor, J.; and Pons, J. 2024. Long-form music generation with latent diffusion. *arXiv preprint arXiv:2404.10301*.

Garcia, H. F.; Seetharaman, P.; Kumar, R.; and Pardo, B. 2023. Vampnet: Music generation via masked acoustic token modeling. *arXiv preprint arXiv:2307.04686*.

He, J.; Liu, J.; Ye, Z.; Huang, R.; Cui, C.; Liu, H.; and Zhao, Z. 2023. RMSSinger: Realistic-Music-Score based Singing Voice Synthesis. In *Findings of ACL*, 236–248.

Ho, J.; Jain, A.; and Abbeel, P. 2020. Denoising Diffusion Probabilistic Models. In *NeurIPS*, volume 33, 6840–6851.

Ho, J.; and Salimans, T. 2022. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*.

Hsu, W.-N.; Bolte, B.; Tsai, Y.-H. H.; Lakhotia, K.; Salakhutdinov, R.; and Mohamed, A. 2021. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM transactions on audio, speech, and language processing*, 29: 3451–3460.

Huang, Q.; Jansen, A.; Lee, J.; Ganti, R.; Li, J. Y.; and Ellis, D. P. 2022. Mulan: A joint embedding of music audio and natural language. *ISMIR*.

Hwang, J.-S.; Lee, S.-H.; and Lee, S.-W. 2025. Hiddensinger: High-quality singing voice synthesis via neural audio codec and latent diffusion models. *Neural Networks*, 181: 106762.

Jiang, Z.; Ren, Y.; Ye, Z.; Liu, J.; Zhang, C.; Yang, Q.; Ji, S.; Huang, R.; Wang, C.; Yin, X.; et al. 2023. Megatts: Zero-shot text-to-speech at scale with intrinsic inductive bias. *arXiv preprint arXiv:2306.03509*.

Kilgour, K.; Zuluaga, M.; Roblek, D.; and Sharifi, M. 2019. Fréchet Audio Distance: A Reference-Free Metric for Evaluating Music Enhancement Algorithms. In *Interspeech*, 2350–2354.

Kim, J.; Kong, J.; and Son, J. 2021. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In *ICML*, 5530–5540.

Kim, T.; and Nam, J. 2023. All-In-One Metrical And Functional Structure Analysis With Neighborhood Attentions on Demixed Audio. In *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*.

Kingma, D. P.; and Welling, M. 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

Kong, J.; Kim, J.; and Bae, J. 2020. Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis. In *Advances in neural information processing systems*, volume 33, 17022–17033.

Kumar, R.; Seetharaman, P.; Luebs, A.; Kumar, I.; and Kumar, K. 2023. High-Fidelity Audio Compression with Improved RVQGAN. In *NeurIPS*, volume 36, 27980–27993.

Lam, M. W. Y.; Tian, Q.; Li, T.; Yin, Z.; Feng, S.; Tu, M.; Ji, Y.; Xia, R.; Ma, M.; Song, X.; Chen, J.; Yuping, W.; and Wang, Y. 2023. Efficient Neural Music Generation. In *NeurIPS*, volume 36, 17450–17463.

Li, Y.; Yuan, R.; Zhang, G.; Ma, Y.; Chen, X.; Yin, H.; Xiao, C.; Lin, C.; Ragni, A.; Benetos, E.; et al. 2024. MERT: Acoustic music understanding model with large-scale self-supervised training. *ICLR*.

Lin, L.; Xia, G.; Jiang, J.; and Zhang, Y. 2023. Content-based controls for music large language modeling. *arXiv preprint arXiv:2310.17162*.

Liu, J.; Li, C.; Ren, Y.; Chen, F.; and Zhao, Z. 2022. Diffsinger: Singing voice synthesis via shallow diffusion mechanism. In *AAAI*, volume 36, 11020–11028.

Loshchilov, I.; and Hutter, F. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.

Luo, Y.; and Yu, J. 2023. Music source separation with band-split RNN. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31: 1893–1901.

Melechovsky, J.; Guo, Z.; Ghosal, D.; Majumder, N.; Herremans, D.; and Poria, S. 2024. Mustango: Toward controllable text-to-music generation. In *NAACL*, 8293–8316.

Peebles, W.; and Xie, S. 2023. Scalable diffusion models with transformers. In *CVPR*, 4195–4205.

Peng, P.; Huang, P.-Y.; Li, D.; Mohamed, A.; and Harwath, D. 2024. VoiceCraft: Zero-Shot Speech Editing and Text-to-Speech in the Wild. In *ACL*, 12442–12462.

Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; and Liu, P. J. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21: 1–67.

Rajbhandari, S.; Rasley, J.; Ruwase, O.; and He, Y. 2020. Zero: Memory optimizations toward training trillion parameter models. In *International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–16.

Rombach, R.; Blattmann, A.; Lorenz, D.; Esser, P.; and Ommer, B. 2022. High-resolution image synthesis with latent diffusion models. In *CVPR*, 10684–10695.

Ronneberger, O.; Fischer, P.; and Brox, T. 2015. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI 2015*, 234–241.

Shen, K.; Ju, Z.; Tan, X.; Liu, Y.; Leng, Y.; He, L.; Qin, T.; Zhao, S.; and Bian, J. 2023. Naturalspeech 2: Latent diffusion models are natural and zero-shot speech and singing synthesizers. *arXiv preprint arXiv:2304.09116*.

Su, J.; Ahmed, M.; Lu, Y.; Pan, S.; Bo, W.; and Liu, Y. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568: 127063.

Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

van den Oord, A.; Vinyals, O.; and kavukcuoglu, k. 2017. Neural Discrete Representation Learning. In *NeurIPS*, volume 30.

Vyas, A.; Shi, B.; Le, M.; Tjandra, A.; Wu, Y.-C.; Guo, B.; Zhang, J.; Zhang, X.; Adkins, R.; Ngan, W.; et al. 2023. Audiobox: Unified audio generation with natural language prompts. *arXiv preprint arXiv:2312.15821*.

Wang, C.; Chen, S.; Wu, Y.; Zhang, Z.; Zhou, L.; Liu, S.; Chen, Z.; Liu, Y.; Wang, H.; Li, J.; et al. 2023a. Neural codec language models are zero-shot text to speech synthesizers. *arXiv preprint arXiv:2301.02111*.

Wang, Y.; Ju, Z.; Tan, X.; He, L.; Wu, Z.; Bian, J.; et al. 2023b. Audit: Audio editing by following instructions with latent diffusion models. In *NeurIPS*, volume 36, 71340–71357.

Wu, S.-L.; Donahue, C.; Watanabe, S.; and Bryan, N. J. 2024a. Music controlnet: Multiple time-varying controls for music generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32: 2692–2703.

Wu, Y.; Chen, K.; Zhang, T.; Hui, Y.; Berg-Kirkpatrick, T.; and Dubnov, S. 2023. Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation. In *ICASSP*, 1–5.

Wu, Y.; Shi, J.; Tang, Y.; Yang, S.; Jin, Q.; et al. 2024b. TokSing: Singing Voice Synthesis based on Discrete Tokens. In *Interspeech*, 2549–2553.

Yang, D.; Wang, D.; Guo, H.; Chen, X.; Wu, X.; and Meng, H. 2024. SimpleSpeech: Towards Simple and Efficient Text-to-Speech with Scalar Latent Transformer Diffusion Models. In *Interspeech*.

Yu, J.; Chen, H.; Bian, Y.; Li, X.; Luo, Y.; Tian, J.; Liu, M.; Jiang, J.; and Wang, S. 2024. AutoPrep: An Automatic Preprocessing Framework for In-The-Wild Speech Data. In *ICASSP*, 1136–1140.

Yu, J.; Chen, H.; Luo, Y.; Gu, R.; and Weng, C. 2023. High Fidelity Speech Enhancement with Band-split RNN. In *Interspeech*, 2483–2487.

Zeghidour, N.; Luebs, A.; Omran, A.; Skoglund, J.; and Tagliasacchi, M. 2021. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30: 495–507.

Zhang, Y.; Cong, J.; Xue, H.; Xie, L.; Zhu, P.; and Bi, M. 2022a. Visinger: Variational inference with adversarial learning for end-to-end singing voice synthesis. In *ICASSP*, 7237–7241.

Zhang, Y.; Ikemiya, Y.; Xia, G.; Murata, N.; Martínez-Ramírez, M. A.; Liao, W.-H.; Mitsufuji, Y.; and Dixon, S. 2024. MusicMagus: Zero-Shot Text-to-Music Editing via Diffusion Models. In *IJCAI*, 7805–7813.

Zhang, Y.; Xue, H.; Li, H.; Xie, L.; Guo, T.; Zhang, R.; and Gong, C. 2022b. Visinger 2: High-fidelity end-to-end singing voice synthesis enhanced by digital signal processing synthesizer. *arXiv preprint arXiv:2211.02903*.

Appendix A: Lyric Preprocessor

Specifically, within this work's context, the song structure is simplified and represented as 6 categories: $[verse]$, $[chorus]$, $[bridge]$, $[intro]$, $[outro]$, $[inst]$, among which $[verse]$, $[chorus]$ and $[bridge]$ indicate the attributes of lyrics (*lyric-related*), while the other tokens describe the non-vocal sections of a song (*accompaniment-only*). To enhance the controllability of structure tokens, these two different types are integrated into the text sequence in different ways, as illustrated in Figure 5.

For lyric-related tokens, an addition operation is applied between the type embedding and the corresponding text similar to Devlin et al. (2018). For accompaniment-only tokens, each entry is repeated according to the duration of the corresponding non-vocal section and directly inserted into the text sequence. In this way, a potential alignment between lyrics and semantic tokens is ensured, as each entry in the text sequence corresponds to a duration within the generated song.

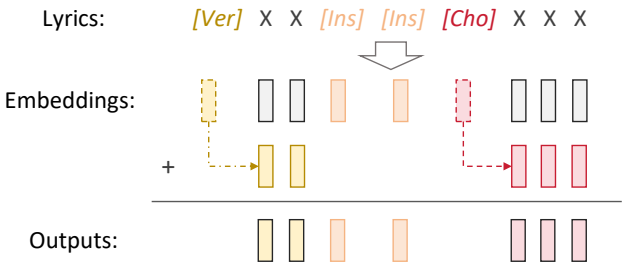


Figure 5: Text token representation for the structure-guided lyrics.

Appendix B: Configuration Details

For the semantic tokenizer, we take the output of the 13th layer of MERT-330M and the last layer of HuBERT as semantic representations. And each RVQ has a codebook size of 10,000. The frame rate of semantic tokens is 25.

To conduct a comprehensive comparison, we trained three different models: SongLM, SongEditor, and SongEditor+. SongLM serves as our baseline and can only generate songs from scratch. SongEditor supports segment-wise editing, while SongEditor+ is capable of both segment- and track-wise editing tasks. All models are based on the Llama architecture (Touvron et al. 2023) with 1024 hidden dimensions and 16 attention heads in each layer with a frame rate of 25. SongLM and SongEditor consist of 16 Llama decoder layers with causal masking. For SongEditor+, an additional cross-attention is inserted into each layer, and the multi-source encoder is a two-layer transformer encoder. Due to computing resource limitations, we reduced the number of layers to 12. Each model contains approximately 800M parameters. The top-k sampling with $k = 250$ and the classifier-free guidance (Ho and Salimans 2022) with a coefficient of 1.5 is employed during inference.

We trained all models using the AdamW optimizer (Loshchilov and Hutter 2017) with a learning rate of 1e-4 for approximately 200K steps. 16 NVIDIA A100 GPUs were used in the training process and the actual batch size is 128. The DeepSpeed ZeRO strategy (Rajbhandari et al. 2020) and bf16 precision were adopted to accelerate the training and reduce memory consumption. Each condition was dropped with a probability of 0.2 during the training phase. For SongEditor+, each source (vocal, accompaniment, or empty embedding) was randomly selected with equal probability in each batch.